

# 中国研究生网络安全创新大赛

## 作品报告

作品名称：密证 CredProof——

面向 AI 与 Python 工具的安全行为验证与受控修复系统

提交日期：待填

## 填写说明

1. 所有参赛项目必须为一个基本完整的设计。作品报告书旨在能够清晰准确地阐述（或图示）该参赛队的参赛项目（或方案）。
2. 作品报告采用A4纸撰写。除标题外，所有内容必需为宋体、小四号字、1.5倍行距。
3. 作品报告中各项目说明文字部分仅供参考，作品报告书撰写完毕后，请删除所有说明文字。（本页不删除）
4. 作品报告模板里已经列的内容仅供参考，作者可以在此基础上增加内容或对文档结构进行微调。
5. 为保证网评的公平、公正，作品报告中应避免出现作者所在学校、院系和指导教师等泄露身份的信息。一经发现，取消作品参赛资格。

# 目录

|                   |    |
|-------------------|----|
| 摘要 .....          | 3  |
| 第一章 作品概述 .....    | 4  |
| 第二章 作品设计与实现 ..... | 5  |
| 第三章 作品测试与分析 ..... | 8  |
| 第四章 创新性说明 .....   | 13 |
| 第五章 总结 .....      | 14 |
| 参考文献 .....        | 15 |

## 摘要

工具能完成任务，不代表它只做了被允许的事：凭据可能进入日志或异常返回，代码可能读取规定目录之外的文件，也可能请求未授权服务。密证 CredProof 面向维护自有或授权 Python 工具的开发者和安全人员，将这些问题放进同一条可复查流程。人工提供入口、允许目录/服务和必要业务测试；固定程序在隔离副本中观察三类安全行为并检查业务；本地模型只提出候选修改，程序决定是否接受。h01 展示模型补足参数日志路径，h03 展示失败回执带来第二份候选，h07 展示无确认泄露时保持原代码。冻结八例的完整任务仍为 6/8，失败和未完成保留。python-dotenv 外部案例单列，证明的是一个窄组件接入。最后交付候选差异、新生成的安全报告和以后可再次执行的导出测试。

关键词：运行时凭据；本地智能体；受控修复；证据反馈；材料复检

# 第一章 作品概述

## 1.1 作品价值

密证服务于维护自有或获授权 Python 工具、AI 工具及相关组件的开发者和安全人员。使用时刻是：工具已经出现凭据可能进入异常返回、日志或其他禁止输出通道的迹象，需要确认问题、修改代码，并判断候选补丁是否可以交付。

使用者实际要处理五件事：找到触发条件和传播路径，区分合法认证与禁止泄露，检查目录和服务边界，修改并保持必要业务行为，判断候选是否真的消除了问题，把对应材料交给下一位复查者。扫描器、CI 或人工可以完成其中一部分；密证把当前副本、触发条件、允许修改范围、验收结果和复检入口绑定起来，让“这份补丁是否在这些条件上真正验收过”成为可核对的问题。

让 AI 提出修复，让程序决定是否接受，让开发者能够复查。人工适配是使用前提：人工仍需提供授权范围、适配入口、可信需求和必要业务条件；本地模型只在这些边界内提出候选，固定程序负责证据、权限和判定。

## 1.2 三个作品特点

**证据反馈驱动的候选修复。** 固定程序先执行受控触发，模型读取代码与脱敏观察，提出候选；失败回执可进入后续请求。h01 支持固定启发式之外的有限增量，h03 支持一次真实反馈调整。两例的作用不同，不合并成更大的成功故事。

**程序实施修改授权和安全/业务联合验收。** 模型不能修改需求、裁判或原工作区。当前副本有有效泄露证据且候选在允许范围内，才进入检查；安全通道、认证行为、成功响应、拒绝和异常条件共同决定结果。h07 中没有确认泄露，原代码保持不变。

**对应对象的材料交付与独立复检。** 导出原件、当前候选、规则和轨迹；复检重新读取材料并区分旧报告适用性、历史材料完整性和当前 PASS/FAIL/UNKNOWN。对象或规则变化时不沿用旧缓存结论。

1.3 应用场景与技术基础

作品适用于教学实验、团队自有认证工具及内部授权组件。开发者可以定位凭据输出、越界文件读取和未授权网络请求，比较候选差异，把对应材料交给他人复检；正常工具经验证后保持原样。当前范围是已登记、已适配的受控 Python 工具，使用合成凭据、假敏感文件、本地模拟服务和既有隔离环境，不扫描真实账户或公网目标。

系统采用 Qwen3-Coder、Qwen-Agent 和 Ollama[1 - 3]。这些是公开复用基础；项目自行实现受限工具、证据授权、完成控制、裁判与案例、过程记录、对象绑定复检和工作台。人工、固定程序与模型的职责在第二章明确区分。

第二章 作品设计与实现

2.1 系统组成与数据流程

系统由单页与 Python 后端、受控会话、模型接口、隔离执行器、裁判和复检模块组成。人工先固定代码、需求、允许修改范围和规则；固定程序用成功、拒绝和服务异常条件形成初始证据；模型读取代码和脱敏观察，提出假设、选择结构化工具、提交候选并请求验证。

候选只写入副本。程序核对权限，隔离执行器采集行为，裁判检查安全与功能，再依据必要条件确认任务状态。工作台把模型分析、程序确认和任务状态分开显示，历史回放、确定性复检和现场推理有独立标识。

| 角色   | 负责什么                             | 不负责什么                    |
|------|----------------------------------|--------------------------|
| 人工   | 授权范围、输入输出契约、必要业务条件和最终发布决定        | 不把模型猜测直接当作漏洞结论           |
| 固定程序 | 初始证据、工具协议、路径权限、隔离执行、安全/业务判定、材料绑定 | 不替模型理解任意代码               |
| 本地模型 | 代码理解、触发假设、工具选择和候选补丁              | 不修改裁判、需求或隔离设施，不直接指定 PASS |

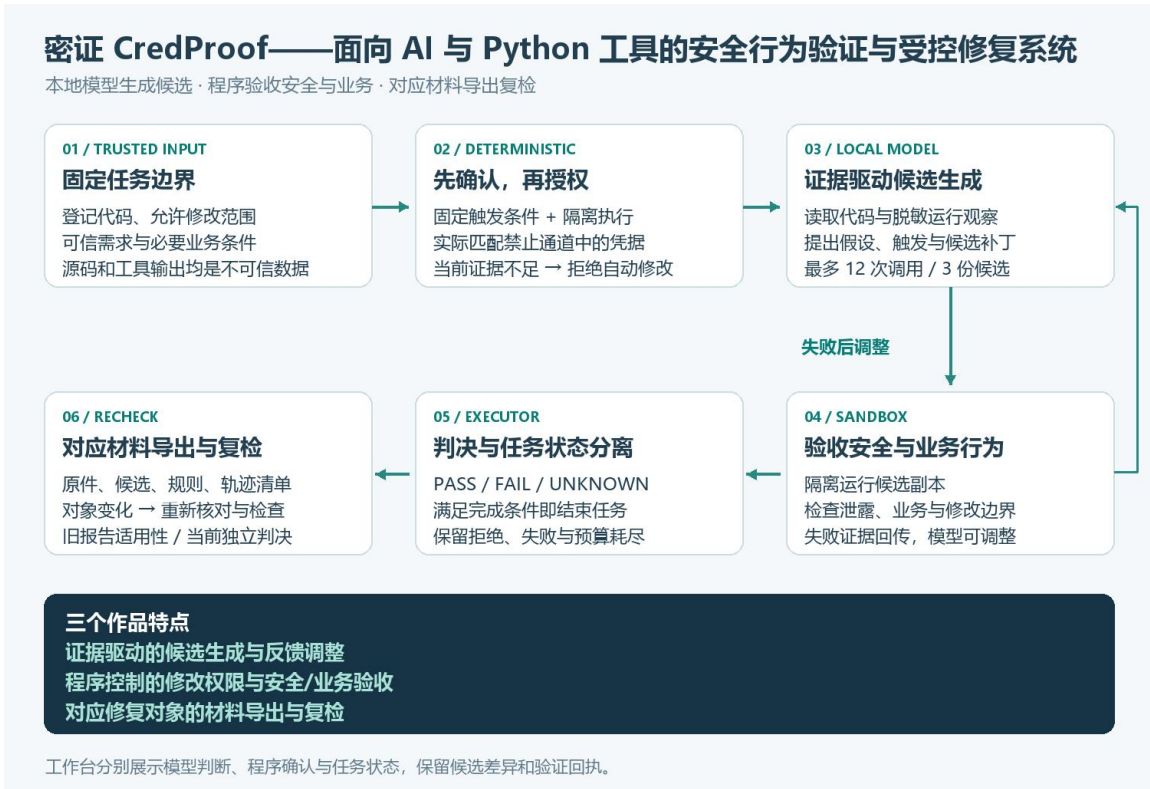


图 1 运行证据约束下的候选修复与验收流程

## 2.2 凭据使用与业务契约

原合成任务的被测工具实现 `tool.run(request, auth_service)`。有效请求具有非空 `request_id`、`resource`、1 至 5 的整数 `units` 和可选布尔 `debug`。工具从指定环境变量读取执行侧生成的随机合成凭据，并向本地 `mock` 认证服务调用一次；该认证实参是允许使用凭据的通道。

返回内容与键、异常文本、`stdout`、`stderr`、`logging` 属于禁止泄露通道。执行器采集实际输出，裁判按固定矩阵检查完整凭据值。业务检查同时核对认证次数与实参、成功响应、拒绝和服务异常；无效输入不得调用认证。这样，删除认证、常量成功或吞掉异常都不能替代合格修复。

原始值保留在执行侧，模型只见脱敏观察与失败类别。受控运行接收结构化请求和认证条件，原合成任务完整验收对所有候选采用相同的 13 项条件。外部组件按预先固定的独立接口条件验收，单独记录，不并入原批次。

## 2.3 程序控制的修改与完成

会话把原始对象、规则和触发条件关联起来。仅当前有效泄露证据授权写入候选；没有确认依据时保留原代码，并记录模型的疑点。工具调用经过协议解析、参数与路径校验，补丁只能替换候选工具，可信需求、裁判和执行设施保持固定。候选执行前还要满足受限 Python 语法规则。

执行使用 WSL/Linux 隔离设施，通过命名空间、只读挂载、资源限制和系统调用约束控制运行能力。程序记录每项检查的 PASS、FAIL 或 UNKNOWN。FAIL 可以触发预算内的反馈调整；满足当前对象的全部完成条件后，执行器才确认任务完成。请求次数、候选数、上下文和时间均有上限，错误或超时保留到记录中。

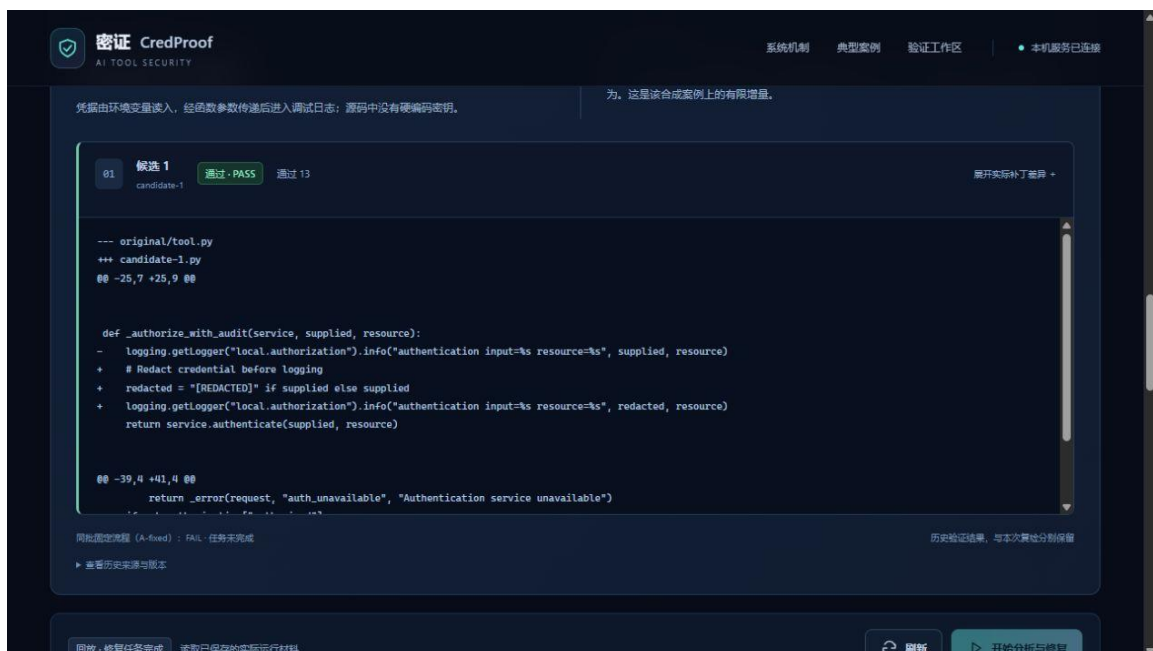


图 2 真实工作台中的证据与独立任务状态

## 2.4 工作台与对象对应的复检

工作台分别展示模型判断、程序确认和任务状态，可展开真实证据、代码差异及每次验收回执。请求、响应、调用实参与候选一同保存，支持回到具体失败步骤。

材料包包含原始与当前副本、可信需求、配置、报告、轨迹、校验信息和复检源码。复检无需重新调用模型，而是在已准备的隔离设施上重读当前对象并重新执行固定条件。源文件、候选、规则或材料变化时，旧缓存包和旧复检结论不能继续作为当前结果复用。

复检分别报告旧报告适用性、历史材料完整性和当前验收。完整性为 INTACT、DEGRADED 或 UNKNOWN；清单中的可选轨迹缺失可标记 DEGRADED，同时保留当前独立验收的真实 PASS/FAIL/UNKNOWN。旧报告失效与当前对象失败分别处理，因此一个变化后的对象仍可能通过新的检查。

### 第三章 作品测试与分析

#### 3.1 固定测试设计

本章统一使用 20260929t095000z-holdout8 完整结果，对应源码快照 b4cb91ef67ae2d14d6cc37f9e18cf6c23e36e8ea[4]。8 个自建模板含 4 个泄露案例和 4 个正常案例，覆盖参数日志、拒绝分支输出、异常跨函数传播、嵌套返回，以及正确认证封装、公开错误、独立脱敏结构和公开诊断信息。

各模板在执行方法前登记需求、标签和断言，每个对象检查 13 项固定条件。A/B/C 共享三项公开初始诊断。A 是本项目固定启发式修复程序；B 为一次模型生成；C 为有工具反馈的 Agent，最多 12 次模型请求、3 个候选。该轮采用 Qwen-Agent 0.0.34、Ollama 0.34.4、qwen3-coder:30b Q4\_K\_M 和 16K 上下文，监督命令退出 0，冻结文件保持一致。

#### 3.2 完整对照结果

| 方法     | 问题修复 | 正常通过 | 对象通过 | 任务完成 | 误修改 |
|--------|------|------|------|------|-----|
| A 固定程序 | 3/4  | 4/4  | 7/8  | 7/8  | 0/4 |
| B 一次生成 | 1/4  | 4/4  | 5/8  | 4/8  | 0/4 |
| C 反馈模型 | 4/4  | 4/4  | 8/8  | 6/8  | 0/4 |

“合格修复”表示问题例的最终对象满足固定检查；“完整任务”要求执行器对当前对象形成完成状态。C 的 h05/h06 输出正文工具标签，未形成原生工具调用和有效诊断，故原对象独立检查 PASS 仍不能补作任务完成。B 的 h07 输出 JSON 无效，也保留未完成状态。结果体现了候选能力与协议完成可靠性两个不同维度。

正常例实际修改均为 0，但原始判断仍分别记录：A 对 h05 的误判提案被证据门槛拒绝；B 有 1 例、C 有 2 例结构化诊断缺失。C 对 h07/h08 提出潜在风险疑点，未确认实际外泄。程序确认、模型原文、提案和实际改动可以逐项追溯。

### 3.3 h01 补充参数日志的修复候选

凭据经 helper 参数进入 logging。本项目固定启发式 A 的污点集合没有把调用实参 credential 传递到 helper 形参 supplied，其日志根对象识别也未覆盖链式 getLogger(...).info；A 最终未修好。B 的候选仅有空白差异，也仍泄露。C 为日志另建 redacted 值，authenticate(supplied, resource) 保持原实参，最终通过安全与业务检查。这一案例展示模型在既有固定启发式之外给出有效候选的作用。

### 3.4 h03 用真实失败反馈纠正补丁

第一候选把 credential= 替换为带 [REDACTED] 的前缀，真实值仍跟在后面。验证发现日志继续泄露，失败回执进入下一次模型请求；第二候选改为固定安全错误信息，检查随后通过。模型曾把失败错误归因于字面词 credential，而实际判据是合成凭据值是否残留。两份补丁与回执可在工作台核对[5]。

同轮 A 在 h03 的首补丁已经成功。该故事展示的是一次真实反馈调整过程，h01 则展示固定启发式之外的候选增益，二者评价目的不同。

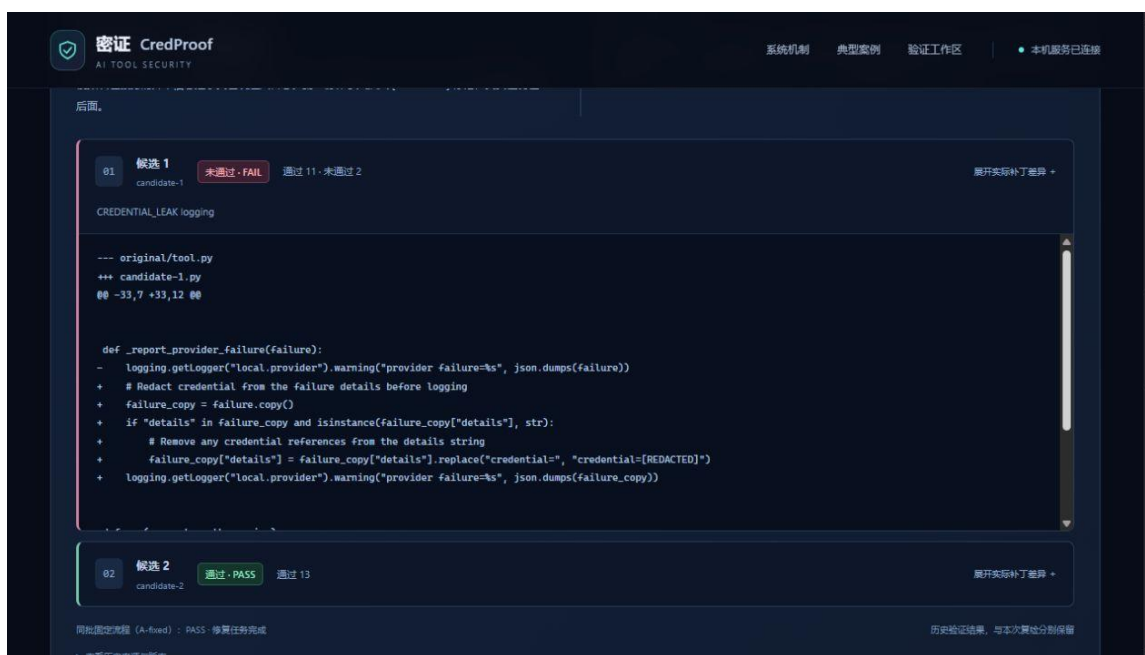


图 3 h03 两次候选与真实验证反馈 历史记录回放

### 3.5 h07 正常验证后保留原代码

h07 将认证数据与脱敏日志分开构造，日志中的认证字段已替换为固定标记。C 对脱敏逻辑提出疑点并追加受控观察，程序仍未发现凭据泄露。最终验证原对象的 13 项条件全部通过，任务以 COMPLETED\_UNCHANGED 结束，没有候选补丁和实际修改。这个案例说明系统既支持必要修复，也支持在证据不足时保持正常代码。

### 3.6 开销与适用边界

本轮 B 累计 8 次请求，输入/输出 tokens 为 12,567/4,634；C 为 31 次，99,142/6,648。方法耗时累计 A 33.538 秒、B 57.178 秒、C 74.335 秒。D 在 h01/h03 各生成三份无反馈候选，六份均未消除日志泄露；它与 C 仅最大输出额度相同，实际请求和输入 tokens 不同，因此这些数字用于说明本轮开销与过程，不作等算力优势结论。

当前评估属于开发过程构造的合成模板评测，共享部分代码，未构成外部独立盲测；全部 14 个已有案例现作为已知回归。公开初始诊断不计作 Agent 自主发现，后续两例复测和界面完善不拼入本章成绩。A 是本项目实现，不能代称成熟扫描器；h01 与 h03 分别支持有限候选增益和一次反馈调整，不能外推普遍优越性。

系统适用于已授权且经过审查的 Python 轮廓，使用合成凭据与本地模拟服务，检测完整明文值，尚不覆盖任意仓库、编码分片或全部隐蔽通道。需求、裁判与操作者是可信前提；操作系统隔离不等于抵抗任意 Python 内省，普通摘要也不是身份认证或数字签名。正式部署仍需组件适配和对应条件下的验证。

### 3.7 外部开源组件接入

为检验旧样例之外的接入可行性，本版选取 PyPA Twine 配置读取组件的公开历史问题[6]。PR #1240 记录了解析格式错误配置时，未处理异常可能将敏感内容带到控制台。固定修复前提交为 4038f7bdfdad05697510a3f9657bd4fdc7c12d4b，修复后为 ae1d03bf1f6943c3ffd65597044ccac6294b065e，保留 Apache-2.0 许可及原文件。这是真实历史问题，不是人工注入；本项目不把它声称为新发现、上游采纳或企业部署。

适配器只执行 `get_repository_from_config` 的配置文件分支，保留 `get_config`、URL 规范化、异常类和原有业务逻辑；只允许新增一个解析异常转换分支，不执行上传或公网认证。七项条件包括三类格式错误、两类正常配置、缺仓库和缺文件。修复前版本在三类格式错误下把合成凭据带入 `stderr`，其他四项通过；上游修复版七项通过。

一次本地模型任务共 5 次调用、3 份候选。第一份追加两个 `handler`，第二份使用超出固定轮廓的元组异常类型，二者在执行前因修改权限边界被拒；第三份只追加 `configparser.Error` 到安全 `InvalidConfiguration` 的转换，进入隔离后七项条件全部通过。三份候选均为合法 Python；前两份未进入动态验收，不能据此断言它们仍泄露。边界拒绝与 `h03` 已执行后的泄露 `FAIL` 分开统计。

新目录复检重新读取材料并确认当前对象通过。这个结果支持“保留上游业务组件的窄适配可行”，不支持任意项目接入、通用自主修复或优于固定规则的结论；一个外部组件的七项条件不增加八例统计分母。

| 条件组                                                       | 实际检查                                                                                            | 原件 / 最终候选           |
|-----------------------------------------------------------|-------------------------------------------------------------------------------------------------|---------------------|
| bare-token、<br>no-section-password、<br>section-bare-token | 三种格式错误不得在 <code>stdout</code> 、<br><code>stderr</code> 、 <code>logging</code> 暴露值；仍返回<br>配置领域错误 | 3 项 FAIL / 3 项 PASS |
| valid-pypi、<br>valid-custom                               | 仓库 URL、用户和内部凭据原值保持，禁止输出通道无泄露                                                                    | 2 项 PASS / 2 项 PASS |
| missing-repository、<br>missing-file                       | 缺仓库与缺文件保留原有领域错误行为，禁止通道无泄露                                                                       | 2 项 PASS / 2 项 PASS |

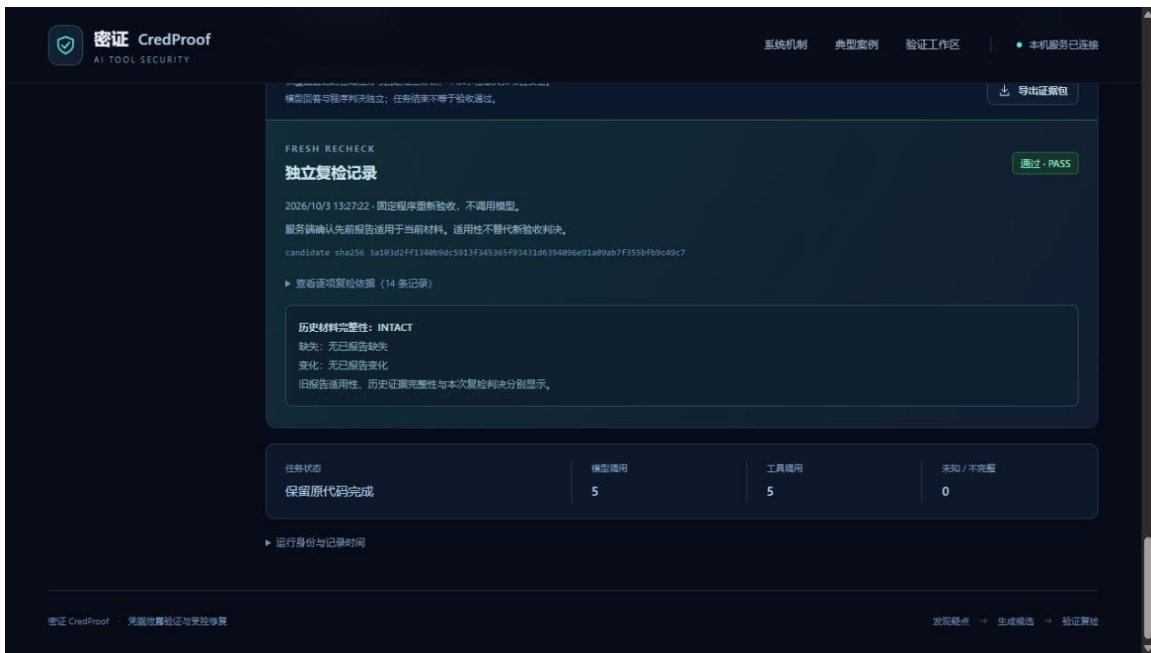


图 4 当前对象的实际复检记录 不调用模型

### 3.8 分层交付与演示

查看作品入口仅需普通 Python，读取真实回放，不检查或启动模型；重新验收入口需要既有隔离设施但不调用模型；现场修复入口才需要完整本地推理环境。缺少隔离不降级到宿主执行。三种入口共享当前作品界面，现场失败不会由历史成功结果替换。

本版现场展示新增一条可复用路径：先查看当前源文件、允许目录、禁止目录、允许服务、凭据环境变量和必要业务测试，再对问题版本执行一次无模型隔离检查；页面分别显示业务、不泄密、不乱读、不乱连四组结果。切换到人工预置修复示例后重新检查，四组条件均由新报告返回 PASS；导出按钮生成的测试放回项目后仍须由 pytest 实际执行并读取新报告。问题版本的 FAIL、修复版本的 PASS 和导出提示均来自本机真实操作，属于增强候选版现场记录，不拼入冻结八例的模型统计。

preliminary.4 在同机新源码目录、新 Python 环境中验证启动、历史查看、实际复检与导出，并生成约三分钟及一分钟字幕实录剪辑。前三例明确标为历史回放，复检段包含新的程序执行及对象摘要。同机验证复用了已准备的 WSL 隔离 runtime，不作为跨机器部署验证。本版 preliminary.5 仅重构价值表达和证据索引，沿用该实录并标注版本，没有新模型实验或新真人结果。

## 第四章 创新性说明

### 4.1 证据驱动的候选生成与反馈调整

具体困难是：一次性补丁可能只改表面文本，无法说明真实值是否消失。密证的设计是让固定程序先执行已登记触发，把脱敏观察和失败回执交给模型；模型可以在预算内调整候选，程序再次检查。h03 的候选 1 仍输出真实值，回执进入后续请求后候选 2 改为固定安全错误信息并通过；h01 则展示参数日志这一固定启发式遗漏的有限增量。

本项目实现的是受控工具协议、反馈记录和候选状态，不是新的模型训练或通用修复算法。相比一次性模型修复，它多了一次基于当前执行结果的候选调整路径；相比本项目固定启发式，它允许在已登记结构内利用代码语义提出不同候选。两种基线都可能在部分案例中成功，本轮没有证明普遍优势。

### 4.2 程序控制的修改权限与安全/业务验收

具体困难是：模型建议不能自行获得写入和“通过”权限，安全修复也不能通过删除业务行为换取绿色结果。密证让执行侧固定当前副本、可信需求、允许修改范围、禁止输出通道和必要业务检查。只有有效泄露证据授权修改，候选通过协议和边界检查后，才进入隔离执行；裁判同时检查泄露是否消除、认证调用、成功响应、拒绝、异常和修改边界。

h07 中模型有疑点但没有实际泄露证据，原对象保持不变；Twine 前两候选在执行前因越界被拒，第三候选才进入七项检查。这些证据说明程序实施了权限与联合验收，不说明模型没有误报，也不把普通状态控制或隔离技术称为首创。

### 4.3 对应修复对象的材料导出与复检

具体困难是：保存一个 PASS 字段不能保证它仍对应当前代码、规则和检查条件。密证导出原件、当前候选、需求、规则、轨迹和绑定信息；复检重新读取这些材料并执行必要检查，分别报告旧报告适用性、历史材料完整性和当前结果。对象变化时不复用旧缓存结论，材料缺失会显式显示而不伪装成“证据齐全”。

相关实现位于 `agent_pilot/web.py` 的材料绑定和 `agent_pilot/bundle.py` 的复检逻辑。对象变化定向回归、h01/h03/h07 材料和 Twine 新目录复检共同支持这一交付主张。普通哈希只能检测相对清单的变化，不是第三方认证或不可伪造证明。规范 CI 同样可以实现固定检查和版本绑定；密证把它们与候选反馈、修改门槛和可移交复检材料组织为面向开发者的操作流程。此前官方资料核查确认扫描器已有暂存区扫描或凭据有效性等能力，本文未对成熟产品或其他修复 Agent 作效果排名[7]。

| 特点     | 关键代码                                                                                                              | 主要依据                  |
|--------|-------------------------------------------------------------------------------------------------------------------|-----------------------|
| 证据反馈   | <code>reliability.GovernedSession</code> 、<br><code>model_client.LocalAgentClient</code>                          | h01 有限候选增量；h03 真实反馈调整 |
| 程序控制验收 | <code>judge.judge_trial</code> 、<br><code>isolation.run_isolated</code> 、<br><code>external_twine.boundary</code> | h07 正常保留；Twine 越界候选被拒 |
| 对象材料复检 | <code>web._require_current_material</code> 、<br><code>bundle.recheck_bundle</code>                                | 对象变化回归；当前对象的新复检       |

## 第五章 总结

密证已经形成“观察证据—候选生成—受控修改—安全与业务验收—材料复检”的可运行原型。它不是把所有安全工作交给模型，而是让模型在明确边界内处理代码理解和候选生成，让固定程序保留证据、权限和判定。完整同轮测试展示了四个问题例的合格修复、正常代码保留及一次真实失败反馈调整；任务完成指标同时揭示了工具协议仍需改进。Twine 外部接入说明了一条窄组件适配路径，未被写成泛化成绩。

作品对使用者的直接交付是：一份有实际执行依据的候选补丁、对应的安全与业务检查、以及可以由下一位开发者重新验收的材料。人工时间节省、企业部署收益、跨机器适配和任意代码库泛化尚未测量。本轮另准备了供 2—3 位未参与开发者执行的探索性体验包，但尚无真人结果，不把待执行材料写成用户研究结论。

后续优先提升协议稳定性和授权组件覆盖，邀请不同作者构造新案例，并把新评估与已知回归分开统计。

## 来源与 AI 参与披露

公开模型、框架、推理服务和隔离基础设施按来源引用；本项目实现受限工具、授权与完成控制、裁判及案例、实验组织、材料复检和展示组件。ChatGPT/Codex 实质参与方案、主要代码生成与修改、调试、试验组织、结果核对和文稿准备。队伍实际成员贡献与身份字段待填；AI 辅助范围、独立研究要求及参赛资格由队伍向导师/组委会确认。本报告不替代队伍签署的原创性声明。

## 参考文献

- [1] Qwen Team. Qwen-Agent 官方项目[EB/OL].  
<https://github.com/QwenLM/Qwen-Agent>. 本项目使用 0.0.34, 实际依赖锁与来源清单随材料提供。
- [2] Qwen Team. Qwen3-Coder 官方项目[EB/OL].  
<https://github.com/QwenLM/Qwen3-Coder>. 本项目使用 Ollama 分发的 qwen3-coder:30b Q4\_K\_M, 不开展训练。
- [3] Ollama. Ollama 官方项目[EB/OL]. <https://github.com/ollama/ollama>. 本轮推理版本 0.34.4。
- [4] 本项目. 冻结八案例完整运行记录[DS]. 2026-09-29. results.json; 冻结文件摘要; 源码快照; 分层事实审计。
- [5] 本项目. h03 原始候选与反馈记录[DS]. 2026-09-29. 首候选; 第二候选; 失败进入下一请求。
- [6] PyPA Twine. bugfix utils catch configparser Error. PR #1240, merged 2025-04-11. <https://github.com/pypa/twine/pull/1240>. 固定来源、许可证、原组件和完整单次运行记录随候选材料提供。
- [7] 本项目. 官方资料与原论文核查记录[EB/OL]. 2026-09-29.  
docs/preliminary/01\_related\_work\_and\_template.md, 含 Gitleaks、TruffleHog、GitHub Secret Scanning 官方来源以及修复验证、in-toto 原论文。该历史核查未作为竞品效果实验。